



**Politecnico
di Torino**

Cybersecurity for Embedded Systems

Trusted Platform Module (TPM)
Implementation

Project Documentation and Manual

Master Degree in Computer Engineering

Referents: Prof. Alessandro Savino, Nicola Scarano

Baraldi Andrea (s339846) - Brogi Filippo (s345969) - Mugnaini Andrea (s338780)

March 11, 2026

Contents

1	Introduction	1
1.1	Abstract	1
1.2	Scope	1
1.3	Common Terms and Acronyms	1
2	TPM System Architecture	3
2.1	Architecture Diagram	3
2.2	TPM Components	3
2.2.1	Cryptographic Primitives	3
2.2.2	Non-Volatile Key Storage	3
2.2.3	Command Chain	5
3	TPM Software Design	6
3.1	Files structure	6
3.2	Data Structures	7
3.2.1	TPM Internal State	7
3.2.2	NV Entry	8
3.2.3	TPM Command Header	8
3.3	Functions	9
3.3.1	Cryptographic Functions	9
3.3.2	Command Handling Functions	9
3.3.3	UART Communication Functions	11
3.3.4	NV Storage Functions	11
3.3.5	General Helper Functions	12
4	The Build process	14
4.1	Build script	14
4.2	QEMU build process	14
4.3	Firmware build process	15
5	TPM Vulnerabilities	16
5.1	Known TPM 2.0 Vulnerabilities	16
5.2	Mitigated Vulnerabilities	17
5.3	Unmitigated Vulnerabilities	17
5.4	Future Work	18
6	TPM Software Testing	19
6.1	Firmware Testing	19
6.1.1	Driving TPM Commands from the Console	19
6.1.2	Checking Response Integrity	19
6.2	Back-End Realization Tests	20
6.2.1	Sanity-Checking NV Allocation	20
6.2.2	Exercising Deterministic Key Creation	20
6.2.3	Round-Trip NV Encryption	21

6.3	Summary	21
7	Programmer's Manual	22
7.1	Memory-Mapped Interface	22
7.2	TPM Command Transmission	22
7.3	Reading TPM Responses	23
7.4	Built-in TPM Commands	23
7.5	Example: Sending TPM2_GetRandom	24
7.6	TPM State Structures	24
7.7	NV Bank Organization	24
7.8	Initialization Sequence	24
7.9	Summary	25
	Bibliography	26

CHAPTER 1

Introduction

1.1 Abstract

This document provides the description of the developed TPM 2.0 (virtualized) hardware device developed for the *Cybersecurity for Embedded Systems* course. It describes system architecture, software design, interface definitions, and the build process. Moreover, a section is dedicated to the user/programmer's manual, providing code examples on how to perform basic TPM functionalities using the implemented features.

1.2 Scope

The project objective was to develop a hardware TPM module, emulating both the target board (NXPS32K3X8EVB) and the TPM module itself through the use of QEMU (1; 2). QEMU (*Quick EMUlator*) is an open-source machine emulator and virtualizer capable of reproducing both hardware (boards, memories, peripherals) and software (firmware or OS environments). The NXPS32K3X8EVB board was emulated through QEMU, while the TPM module was implemented as a custom device model within its hardware abstraction layer.

The TPM (Trusted Platform Module) follows the specification defined by the Trusted Computing Group (TCG) (3; 4; 5). Our implementation references the TPM 2.0 command architecture (Part 3), internal data structures (Part 2), and provisioning recommendations from the TCG guidelines (6). Cryptographic primitives (AES, SHA-256, RSA, ECC) were implemented and tested using the OpenSSL library (7; 8).

1.3 Common Terms and Acronyms

TPM Trusted Platform Module A hardware security module defined by the Trusted Computing Group (TCG) providing secure key storage, attestation, and cryptographic operations (9).

TCG Trusted Computing Group The industry consortium responsible for defining open standards for trusted computing technologies, including the TPM 2.0 specification (3).

TPM 2.0 The current version of the Trusted Platform Module specification, introducing hierarchical authorization, flexible algorithm support, and enhanced cryptographic structures (4).

TPM Command Chain (CC) Logical flow used by TPM to process commands, including validation, authorization, and execution phases defined in the TPM Part 3 specification (5).

TPM_RC TPM Return Code Numeric values returned by TPM commands to indicate success or specific failure conditions during command execution.

NV Non-Volatile Memory Persistent storage used by TPM for saving key data, authorization values, and platform configuration registers (PCRs) across reboots.

- TPM2_NV_DefineSpace** TPM command used to define and allocate a new NV index in non-volatile storage (5).
- QEMU Quick EMUlator** A generic, open-source machine emulator and virtualizer used in this project to emulate TPM hardware devices and firmware interactions (1; 2).
- QOM QEMU Object Model** The object-oriented framework used by QEMU to represent devices, buses, and system components as class-based objects (2).
- QAPI QEMU API** Interface used by QEMU for communication between its internal components, monitor, and device frontends.
- EVP Envelope API (OpenSSL)** High-level cryptographic interface in OpenSSL used to perform digest, encryption, and signature operations abstractly (8).
- KDFa Key Derivation Function (Type a)** Function defined in the TPM specification used to derive cryptographic keys through HMAC-based counter mode (5).
- CFB Cipher Feedback Mode** AES encryption mode used in TPM for symmetric encryption/decryption (AES-CFB-128) (8).
- RSASSA / RSAPSS** RSA signature schemes supported by TPM for signing operations (PKCS#1 v1.5 and Probabilistic Signature Scheme, respectively).
- Glib** Core C library used by QEMU for memory management, data structures, and event handling (2).
- SITL Software In The Loop** Simulation setup used to test embedded firmware (such as the TPM device model) entirely in software.
- Make / Meson / Ninja** Build systems used in QEMU and firmware development to compile and link project components efficiently.
- EVP_PKEY** OpenSSL structure representing a private or public key, used for TPM RSA and ECC key tests (8).
- TPMA_NV** TPM attribute bitfield structure describing permissions and access behavior for NV indexes (4).

CHAPTER 2

TPM System Architecture

2.1 Architecture Diagram

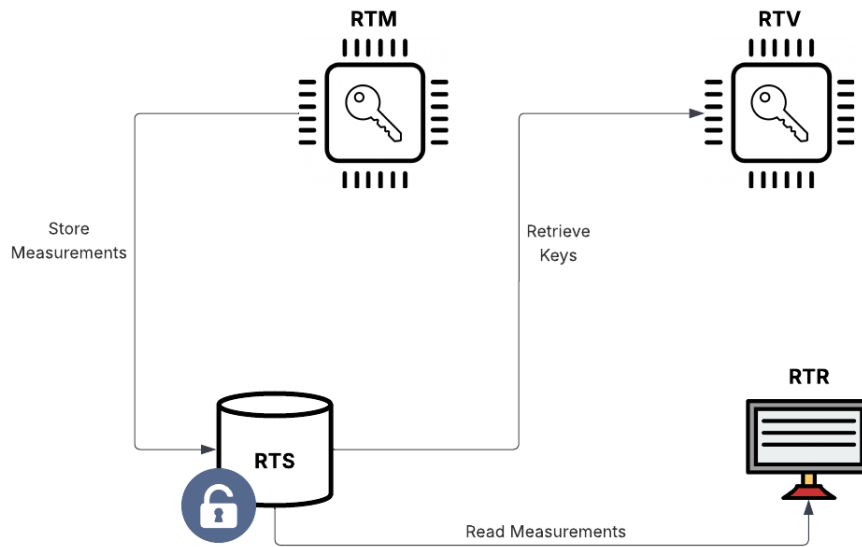


Figure 2.1: TPM Architecture and Roots of Trust

2.2 TPM Components

2.2.1 Cryptographic Primitives

In order to ensure secure key storage, the TPM module's firmware needs to implement some cryptographic functions of many kinds (hashes, asymmetric key generation, symmetric key generation, signing and verifying keys, random number generation)

2.2.2 Non-Volatile Key Storage

The TPM stores internally, in its Non-volatile key storage, data, encapsulated in *Nv Entries*. This includes items such as authorization secrets, monotonic counters, and long-term key material

In this software model, the same concept is represented by a private memory region called the *NV bank*.

Function. NV storage is a crypted space in memory, divided into data entries. Each entry inside it is identified by an index number (`TPM_NV_INDEX`) and is wrapped inside a `NV_Entry` data structure

Table 2.1: Internal NV Entry Structure (**NVEntry**)

Field	Type/Size	Description
pub	TPMS_NV_PUBLIC	The public attributes and metadata for this entry.
authValue	up to 64 bytes	Authorization secret for this NV index.
name	up to 64 bytes	Cached name derived from nameAlg .
data	pointer	Address of the actual NV data buffer in the NV bank.
dataLen	16 bits	Length of data area (equals dataSize).
iv_ptr	16 bytes	Initialization vector for encryption/decryption.
written	boolean	True once the index has been written.
readLocked	boolean	Prevents reads until cleared.
writeLocked	boolean	Prevents writes until cleared.

Integration into QEMU. The TPM2 device is implemented as a subclass of **SysBusDevice**, giving it a private address space and memory regions that can be connected directly to the system bus

Table 2.2: Integration of NV Storage Fields within **TPM2State**

Field	Type	Description
nv_bank_mem	MemoryRegion	Private block of host RAM that emulates the TPM’s internal non-volatile storage.
nv_bank_ptr	uint8_t *	Pointer to the start address of the NV memory region, used for direct data access.
nv_map	GHashTable *	Lookup table mapping each defined NV index (TPM_NV_INDEX) to its associated NVEntry object.
nv_alloc_offset	uint32_t	Offset pointer indicating the next available free region inside the NV bank for new NV entries.

The NV bank behaves much like a miniature file system managed entirely by the TPM logic.

Security and data handling. In a real device, the bytes written to NV memory are never stored in the clear, and so does our implementation: All data is written and read as encrypted **AES-CFB** ciphertext, using the symmetric primitives provided by OpenSSL

Each NV index (*NV storage region*) in the storage space is set to a 128-bit initialization vector (IV) to use with AES-CFB.

On NV definition, an entry is created with space for an IV and the encrypted data. For each NV write operation, a random IV is generated for that entry and encrypts the input plaintext into the NV buffer using AES-CFB, while the IV is stored with the ciphertext in the NV state. On read, the stored IV is used to decrypt the data.

CFB encryption produces the same length for the ciphertext as the plaintext (up to the defined *dataSize*)—there is no padding overhead or alignment requirement. As NV indices may not be evenly written and are not in general multiples of the AES 128-bit block size, using CFB mode allows the emulator to encrypt or decrypt any length of data in-place without special handling for partial blocks

Per-Entry IV Usage. Separate IVs for each NV index allow us to encrypt each index differently and independently from the others. Even if two entries have the same plaintext, their ciphertexts will differ due to distinct random IVs, providing additional redundancy and security against **identical-plaintext detection and cross-entropy attacks**. The IV is re-randomized on each write to an

entry (and stored persistently), preventing IV reuse — a known weakness in feedback modes such as CFB and CBC

Reading would simply perform the reverse operation. This ensures that two identical data blocks produce different ciphertexts and that tampering with the stored data can be detected.

Relationship to primary keys. Primary keys created by the `CreatePrimary` command normally exist only in volatile memory

Summary. The NV subsystem forms the TPM's long-term memory. It offers a place to hold sensitive data, protects it through encryption, and allows selected TPM objects to survive resets. Even in an emulated setting, this part of the design is essential for reproducing the behaviour of real-world TPM hardware

2.2.3 Command Chain

The TPM module needs to communicate with the platform (mainly the CPU) and does so by using a **TPM Command Protocol**, which defines what kinds of commands the TPM can execute and which protocol to use to transmit data between the TPM module and the CPU. Common protocols supported by TPM modules are I²C and SPI. The proposed implementation implements UART for simplicity.

CHAPTER 3

TPM Software Design

3.1 Files structure

```
temp-qemu/
├── firmware/
│   ├── test/
│   │   ├── cc/
│   │   ├── crypto/
│   │   ├── nv/
│   │   ├── uart/
│   │   └── test.h
│   ├── build_and_run.sh
│   ├── main.c
│   ├── makefile
│   ├── startup.s
│   ├── uart.c
│   └── uart.h
├── qemu/
│   ├── hw/
│   │   └── tpm/
│   │       ├── Kconfig
│   │       ├── meson.build
│   │       ├── tpm_cc.c
│   │       ├── tpm_crypto.c
│   │       ├── tpm_hw.c
│   │       ├── tpm_nv.c
│   │       └── ...
│   └── include/
│       └── tpm/
│           ├── TPM2_NV_STRUCTS.h
│           ├── tpm2_base_types.h
│           ├── tpm2_cc.h
│           ├── tpm2_crypto.h
│           ├── tpm2_device.h
│           ├── tpm2_marshall.h
│           ├── tpm2_nv.h
│           ├── tpm2_persistent.h
│           ├── tpm2_rc.h
│           └── ...
```

- **temp-qemu/**: The root directory for the project.
- **firmware/**: Contains all source code intended to run on the emulated target board (NXPS32K3X8EVB).

- **test/**: Holds the testing framework (**test.h**) and all test suites for different modules (Command Chain, crypto, NV, UART).
- **build_and_run.sh**: The main script to compile both QEMU and the firmware, and then run the emulation.
- **main.c**: The main entry point for the firmware; it contains the user menu and logic to send commands to the TPM device.
- **uart.c / uart.h**: Firmware-side drivers for UART communication, used for **printf**.
- **qemu/**: Contains the source code for the QEMU emulator itself.
- **hw/tpm/**: This directory holds the custom QEMU device model implementation for our TPM

3.2 Data Structures

3.2.1 TPM Internal State

Listing 3.1: TPMState structure

```

1 struct TPM2State
2 {
3     SysBusDevice parent_obj;
4
5
6
7     MemoryRegion mmio;
8     qemu_irq      irq;
9
10    //NV Storage Memory region
11
12    MemoryRegion nv_bank_mem;
13    uint8_t      *nv_bank_ptr;
14    uint32_t      nv_bank_size;
15    uint32_t      nv_alloc_offset;
16
17
18    /* In-RAM index of entries (not serialized as-is; we pack to nv_bank_ptr) */
19    GHashTable    *nv_map;      // key: GUINT_TO_POINTER(nvIndex) -> NvEntry*
20    uint32_t      nv_count;
21    bool          nv_dirty;     // needs flush to bank flag
22
23    //End NV Storage Memory region
24
25    uint32_t ctrl;
26    uint32_t status;
27    uint32_t key_generated;
28    RSA      *rsa_key;
29    uint8_t  random_data[32];
30
31    uint8_t command_buffer[1024];
32    uint8_t response_buffer[1024];
33    uint32_t cmd_size;
34    uint32_t resp_size;
35    uint32_t resp_idx;
36    enum tpm_state state;
37    enum tpm_operational_mode tpm_mode;
38    //Primay handle sensitive data

```

```

39  uint8_t pps[32], sps[32], eps[32]; //Seeds
40  uint8_t phProof[32], shProof[32], ehProof[32]; //Storage root proofs
41
42  TPM2B_SENSITIVE primary_sensitive; // last created sensitive area
43  TPM2B_PUBLIC    primary_public;    // last created public area
44  TPM2B_NAME      primary_name;      // last created Name
45
46  uint32_t next_transient_handle;
47  bool initialized;
48
49  };

```

This structure models the internal state of the TPM device inside QEMU, including command marshalling registers, Non-volatile indexes, memory and primary seeds and proofs and lastly generated master sensitive. It acts as the core runtime context for command parsing, execution, and response delivery.

3.2.2 NV Entry

Listing 3.2: NVEntry structure

```

1  typedef struct NVEntry {
2      TPMS_NV_PUBLIC pub;          /* nvIndex, nameAlg, attributes, authPolicy,
                                   dataSize */
3      uint8_t        authValue[NVENTRY_MAX_AUTH];
4      uint16_t        authValueLen; /* hash size of nameAlg; enforced at DefineSpace
                                   . */
5
6      /* cached Name of the index (size derived from nameAlg) */
7      uint8_t  name[64];
8      uint16_t nameLen;
9
10     /* backing data */
11     uint8_t *data;          /* length == pub.dataSize */
12     uint16_t dataLen;
13
14     /* mirrors of dynamic bits */
15     bool written;           /* TPMA_NV_WRITTEN (set after first init/write)
16     */
17     bool readLocked;        /* TPMA_NV_READLOCKED      clears on Startup(
18     CLEAR) */
19     bool writeLocked;       /* TPMA_NV_WRITELOCKED semantics per spec */
20
21     /* simple integrity (optional): HMAC over data using a TPM-internal key, if
22     you want */
23     uint8_t *integrity;     /* optional blob */
24     uint16_t integrityLen;
25 } NVEntry;

```

Represents a non-volatile storage object defined by `TPM2_NV_DefineSpace`. It stores public metadata (attributes, size, policy), authorization data, the NV Name, and the backing encrypted data buffer, along with runtime lock and write flags. This acts as the TPM's persistent key-value store entry.

3.2.3 TPM Command Header

Listing 3.3: TPM_CMD_HEADER structure

```

1  /* TPM Command Header */
2  typedef struct {
3      UINT16 tag;           /* TPM_ST_NO_SESSIONS or TPM_ST_SESSIONS */
4      UINT32 size;          /* Total command size in bytes */
5      UINT32 code;          /* TPM_CC command code */
6  } TPM_CMD_HEADER;

```

Header prefix prepended to every TPM command. It identifies whether sessions are present, the byte length of the command buffer, and the TPM command code to dispatch. Used by the command dispatcher to validate and route incoming requests.

3.3 Functions

3.3.1 Cryptographic Functions

Here are the main cryptographical functions implemented by the TPM module

- `static void tpm2_generate_random (TPM2State *s)`: Generates pseudo-random bytes within the TPM state using OpenSSL `RAND_bytes()`
- `int TPM2_SHA256(const uint8_t *data, size_t len, uint8_t out[32])`: Computes the SHA-256 digest of the `data` message with length `len`, and writes it to the 32-bytes output buffer `out`. Returns 1 on success or 0 on failure.
- `int TPM2_KDFa(const EVP_MD *hash, const uint8_t *hmacKey, size_t hmacKeyLen, const char *label, uint32_t bits, uint8_t *out)`: Implements the TPM 2.0 Key Derivation Function (KDFa) as defined in Part 3 of the specification
- `TPM_RC tpm2_CreatePrimary (TPM2State *s, uint32_t primaryHandle, const TPM2B_SENSITIVE_CREATE *inSensitive, const TPM2B_PUBLIC *inPublic, const TPM2B_DATA *outsideInfo, const TPML_PCR_SELECTION *creationPCR, TPM_HANDLE *objectHandle, TPM2B_PUBLIC *outPublic, TPM2B_NAME *name)`: Creates a primary ECC object for the selected hierarchy, deriving a deterministic P-256 key from the hierarchy seed via `TPM2_KDFa` and computing the public point with OpenSSL `EC_POINT_mul()`
- `int TPM2_AES_CFB_Crypt (const uint8_t *key, int keylen, const uint8_t *iv, const uint8_t *in, size_t inlen, uint8_t *out, int enc)`: Encrypts (or decrypts an already-encrypted) the input message `in` of length `inlen`, using `key` as a buffer which contains the key to perform the operation, and its length is specified by `keylen`
- `static void tpm2_generate_rsa_key (TPM2State *s)`: Generates a 2048-bit RSA key pair using OpenSSL primitives (`RSA_new()`, `BN_new()`, and `RSA_generate_key_ex()`) and stores it in `s->rsa_key`. Used for basic `CreatePrimary` operations that rely on RSA rather than ECC
- `int TPM2_RSA_Sign (EVP_PKEY *pkey, const uint8_t *msg, size_t msglen, uint8_t *sig, size_t *siglen, const EVP_MD *md)`: Signs the message pointed by `msg` (length `msglen`) using the RSA private key `pkey` using the hashing algorithm `md`. Writes the output inside `sig` and its length into `siglen`. Returns 1 on success or 0 on failure.

3.3.2 Command Handling Functions

These functions implement the command-processing logic defined in TPM 2.0 Part 3

Command Unmarshalling

- `Unmarshal_UINT8 / Unmarshal_UINT16 / Unmarshal_UINT32 / Unmarshal_UINT64`: Low-level big-endian integer readers used to extract fixed-width TPM fields from a byte stream. Validate buffer bounds, update byte counters, and return TPM codes such as `TPM_RC_SUCCESS` or `TPM_RC_INSUFFICIENT`. These are the primitive decoding functions underpinning all higher-level unmarshalling logic.
- `Unmarshal_GetRandom(const UINT8 *buffer, UINT32 bufferSize, UINT16 *bytesRequested, UINT32 *bytesRead)`: Parses the `bytesRequested` field for `TPM2_GetRandom`. Thin wrapper around `Unmarshal_UINT16`; ensures the command body requests a valid random byte count before execution.
- `Unmarshal_CreatePrimary(const UINT8 *buffer, UINT32 bufferSize, CreatePrimary_Params *params, UINT32 *bytesRead)`: Extracts the full `TPM2_CreatePrimary` parameter structure, including the parent handle, `TPM2B_SENSITIVE_CREATE` parts (userAuth and sensitive data), and size-prefixed public template blob. Leaves detailed `TPMT_PUBLIC` decoding as a future extension, enabling a staged bring-up of key-creation support.
- `Unmarshal_NV_DefineSpace(const UINT8 *buffer, UINT32 bufferSize, NV_DefineSpace_Params *params, UINT32 *bytesRead)`: Decodes `TPM2_NV_DefineSpace` structures: provision handle, authorization buffer, NV public area fields (index, nameAlg, attributes, policy), and data size. Logs decoding steps for debugging and validates size-bounded `TPM2B` areas before NV index creation logic.
- `Unmarshal_NV_Write(const UINT8 *buffer, UINT32 bufferSize, NV_Write_Params *params, UINT32 *bytesRead)`: Parses NV write parameters including authorization handle, NV index, `TPM2B_MAX_NV_BUFFER` payload, and byte offset. Rejects session-authenticated variants (nonzero `authSize`) to keep early prototypes simple while preserving TPM compliance semantics.
- `Unmarshal_NV_Read(const UINT8 *buffer, UINT32 bufferSize, NV_Read_Params *params, UINT32 *bytesRead)`: Extracts NV read arguments: authorization handle, index, requested byte count, and offset. Enforces no-session mode during initial emulator bring-up and logs unmarshalling boundaries, providing a clear trace into NV read staging.

Command Processing

- `static void build_error_response (TPM2State *s, TPM_RC rc)`: Constructs a minimal TPM response packet containing only a header with error code `rc`. Used by all command handlers to standardize failure returns in compliance with TPM 2.0 Part 3 formatting
- `static void build_success_response (TPM2State *s, UINT32 bodySize)`: Generates a full TPM response header indicating `TPM_RC_SUCCESS`, appends a variable-length response body, and ensures 4-byte alignment through zero-padding. Marks the device state as ready to transmit the result to the guest.
- `static void handle_GetRandom (TPM2State *s, const UINT8 *cmdBody, UINT32 bodySize)`: Processes the `TPM2_GetRandom` command by unmarshalling request parameters, generating entropy with `RAND.bytes()`
- `static void handle_CreatePrimary (TPM2State *s, const UINT8 *cmdBody, UINT32 bodySize)`: Simplified `TPM2_CreatePrimary` handler for ECC P-256 primary storage key objects. Unmarshals parameters, generates a new key pair with `tpm2_generate_primary()`, and returns a minimal but valid TPM response structure containing a transient object handle. This handler serves as a proof-of-concept for integrating full key creation in the command chain.

- `static void handle_NV_DefineSpace (TPM2State *s, const UINT8 *cmdBody, UINT32 bodySize)`: Implements the TPM2_NV_DefineSpace command, unmarshalling all NV parameters and calling `tpm2_nv_define_space()`
- `static void handle_NV_Read (TPM2State *s, const UINT8 *cmdBody, UINT32 bodySize)`: Implements the TPM2_NV_Read command. Unmarshals the NV read parameters, validates the NV index, enforces bounds and lock checks, and rejects access to uninitialized indices. Reads ciphertext from the NV bank, decrypts it with the primary seed-derived AES key, marshals the plaintext into the response buffer, and returns the data to the caller. This handler verifies the confidentiality and integrity of NV data and ensures compliance with NV read semantics.
- `static void handle_NV_Write (TPM2State *s, const UINT8 *cmdBody, UINT32 bodySize)`: Implements the TPM2_NV_Write command. Parses command parameters, locates the target NV index, and enforces offset and size constraints as well as authorization hierarchy checks. Invokes the backend `TPM2_NV_Write()` routine to encrypt and store the payload in the NV bank. Marks the index as written and signals a modified NV state. This handler provides authenticated and encrypted persistence for NV data, following TPM-defined NV authorization and write rules.
- `static void tpm2_process_command (TPM2State *s)`: Central TPM command dispatcher. Validates incoming command headers via `TPM2_ValidateCommandHeader()` and routes execution to the correct handler (`GetRandom`, `CreatePrimary`, or `NV_DefineSpace`). Builds structured responses or standardized error codes in compliance with the TPM 2.0 command protocol.

3.3.3 UART Communication Functions

- `void UART_putc(const char c)`: Polls until UART buffer is not full (`!UART_F_RXFF`), then writes the character `c` into the UART data register `UART0_DATA`.
- `char UART_getc(void)`: Polls until UART buffer is not empty (`!UART_F_RXFE`), then returns the character stored into the `UART0_DATA` register.
- `void UART_printf (const char *fmt, ...)`: Mimicks C's standard `printf()` function, handling special character modifiers starting with the character `'%'` and prints the formatted string `fmt` to the output terminal.

3.3.4 NV Storage Functions

- `static NVEEntry *nv_entry_alloc (TPM2State *s, TPM_NV_INDEX index, uint16_t dataSize, uint16_t nameAlg, TPMA_NV attrs, const void *auth)`: Allocates and initializes a new NV storage entry within the TPM's internal non-volatile (NV) memory bank. The function first verifies that the NV region (`s->nv_bank_ptr`) is valid and has sufficient free space to host the requested index. Each NV entry reserves a 16-byte IV region followed by the data payload, both zero-initialized for security. The total allocation size (`16 + dataSize`) is carved from the NV bank at the current allocation offset, which is then advanced accordingly. The function populates `NVEEntry` fields including index, attributes, name algorithm, and access flags (`written`, `readLocked`, `writeLocked`) set to false. The IV and data regions are cleared before use to avoid leakage of stale contents. Returns a pointer to the allocated `NVEEntry` on success, or `NULL` if the NV bank is uninitialized or full. Note that the current implementation logs an error instead of returning a TPM-standard error code (`TPM_RC_NV_SPACE` would be the correct one).

- `TPM_RC tpm2_nv_define_space(TPM2State *s, TPMI_RH_PROVISION authHandle, const TPM2B_AUTH *auth, const TPM2B_NV_PUBLIC *public)`: Allocates a new NV index (NV memory region) and registers it into tpm’s internal state `s`. `public` describes NV index’s metadata and `authHandle` specifies command authorizations (allowed attributes). Authorization data is passed through `auth`. Returns the TPM RC `TPM_RC_SUCCESS` if successful, else another RC indicating failure.
- `static inline void nv_entry_free (NVEEntry *e)`: Releases the memory of a previously allocated NV entry. If the input pointer is `NULL`, the function exits silently. Since NV entry data belongs to a fixed region within the TPM’s NV bank, only the metadata structure allocated by `g_malloc0()` is freed using `g_free()`. The data pointer is explicitly nulled to avoid dangling references but not deallocated, as its ownership remains within the NV memory bank.
- `void tpm2_nv_init (TPM2State *s)`: Initializes the TPM’s Non-Volatile (NV) subsystem by creating a GLib hash table mapping NV indices to their corresponding `NVEEntry*`. Each entry in this table represents an allocated NV slot. The table is configured to automatically call `nv_entry_free()` upon element removal, ensuring clean memory management. The function sets counters such as `nv_count`, resets allocation offsets, and initializes the NV bank to `TPM2_NVSTORAGE_SIZE`. Returns silently if called multiple times or if `s` is `NULL`. Successful initialization is confirmed via debug logging.
- `void tpm2_nv_cleanup (TPM2State *s)`: Destroys the TPM’s NV subsystem, freeing all dynamically allocated NV entries and clearing the internal mapping table. The function first verifies that the `TPM2State` pointer and `nv_map` are valid, then removes and destroys all entries with `g_hash_table_destroy()`. Finally, it resets counters and state flags (`nv_count`, `nv_dirty`) to zero and logs the cleanup completion. Serves as the counterpart to `tpm2_nv_init()` and is called during TPM reset or device shutdown.
- `TPM_RC TPM2_NV_Write(TPM2State *s, TPMI_RH_PROVISION authHandle, TPM_NV_INDEX nvIndex, const TPM2B_MAX_NV_BUFFER *data, UINT16 offset)`: Encrypts and writes plaintext data into the TPM’s NV memory bank using AES in CFB-128 mode. The function checks boundary conditions to ensure the data fits within the NV entry, then generates a fresh 16-byte initialization vector (IV) per entry via `RAND.bytes()`
- `TPM_RC nv_read_decrypt_from_bank (TPM2State *s, NVEEntry *e, uint8_t *out, uint16_t len, uint16_t offset)`: Decrypts ciphertext stored in an NV entry and restores the original plaintext into the provided output buffer. The function validates parameters and NV bounds, then performs AES/CFB decryption using the same symmetric key and IV associated with the entry. Internally calls `TPM2_AES_CFB_Crypt()` with the encrypt flag set to zero

3.3.5 General Helper Functions

These support QEMU integration and device realization

- `static void tpm2_mmio_write (void *opaque, hwaddr addr, uint64_t value, unsigned size)`: Processes guest writes to TPM MMIO registers. Writing to the control register triggers receive or reset states; writing to `TPM2_DATA_REG` appends 32-bit words to the command buffer in big-endian order. When the guest writes 1 to `TPM2_CMD_REG`, the assembled command is executed through `tpm2_process_command()`. Incorrect states or buffer overflows are logged and rejected.
- `static void tpm2_realize (DeviceState *dev, Error **errp)`: Device realization function for QEMU integration. Initializes IRQs, MMIO regions, and NV storage, then performs built-in TPM self-tests: `tpm2_test_definespace()`, `tpm2_test_CreatePrimary()`, and

`tpm2_test_nv_encrypt_decrypt()`. Logs hardware-like initialization details to verify proper TPM emulation setup.

- `static void tpm2_reset (DeviceState *dev)`: Resets the TPM2 device by clearing command and response buffers, zeroing random data, freeing RSA keys, and setting the state machine back to `TPM_STATE.IDLE`. Used during soft resets and control register deassertions.
- `static void tpm2_init (Object *obj)`: Initializes a new TPM2 device instance. Provides the object with default memory mappings and internal structures. This lightweight constructor complements the more detailed configuration in `tpm2_realize()`.
- `static void tpm2_class_init (ObjectClass *klass, void *data)`: Registers TPM2 device class attributes within QEMU's object model, linking the reset routine, VM state description, and realize function. Sets device description to "TPM 2.0 custom device" for debugging visibility
- `static void tpm2_register_types (void)`: Final registration hook that inserts the TPM2 type into QEMU's device registry using `type_register_static()`. Ensures that the TPM2 emulated device can be instantiated via the QEMU system bus at runtime

CHAPTER 4

The Build process

4.1 Build script

The whole build process is handled by a comprehensive script `firmware/build_and_run.sh`, which first compiles QEMU using Meson+Ninja, and then compiles the firmware to run on the emulated board via makefile. QEMU and firmware are separated in different directories, starting from project root: `firmware` and `qemu` respectively. Their respective build processes are also independent of each other

4.2 QEMU build process

The first portion of the `firmware/build_and_run.sh` scripts does exactly this:

```
#####  
#### Build QEMU via Meson + Ninja ####  
#####  
cd ..  
mkdir qemu/build  
cd qemu/build/  
# Configure Meson, get host architecture and OS, ...  
../configure --target-list=arm-softmmu  
# Build QEMU  
make
```

The first step of the process is to call the `configure` script, which detects automatically, based on defined runtime symbols:

- Host machine architecture (x86_64, arm, aarch64, riscv64, ...).
- Host machine OS (Linux, macOS, Windows, FreeBSD).
- Host machine toolchain binaries (compiler, assembler, linker).
- Libraries and dependencies (OpenSSL, Glib, fdt, ...)

Next step is to actually build QEMU itself, along with all of its emulated hw devices (along with the target TPM module).

Since QEMU version 8.x, QEMU itself is built using Meson files, organized in a hierarchical structure as follows

- `qemu/hw/<device>/meson.build`: Meson file for the hw-device `device`, which includes device-specific source files (targets) to the global list of files to compile generated by `qemu/meson.build`.

Then, the main `meson.build` produces a single `qemu/build/build.ninja` file which contains a set of build rules to build each file needed for each QEMU hw device.

4.3 Firmware build process

A second section (shown below) of the build script is dedicated to this task, which uses the makefile `firmware/makefile` to configure the environment and firmware building process.

```
#####  
#### Build firmware via Makefile ####  
#####  
cd ../../firmware  
# Compile and link into .elf file to flash  
make all  
# Run QEMU binary generated by QEMU build to load firmware  
make qemu_start
```

- **make all**: Configures an embedded ARM toolchain (`arm-none-eabi-*`) along with compiler and linker flags and installs required build dependencies on the host system.
- **make qemu.start**: Configures QEMU flag to define the target board (`-M nxps32k3x8evb`) and runs the generated QEMU binary (produced by the QEMU build process inside `qemu/build/` directory) and flash into its memory the `.elf` file for executing the firmware

CHAPTER 5

TPM Vulnerabilities

This chapter describes known TPM 2.0 vulnerabilities, issues found in our implementation, and possible future improvements to increase the security and robustness of the system.

5.1 Known TPM 2.0 Vulnerabilities

Over the years, several vulnerabilities have been found in TPM 2.0 devices and software implementations. Most of these issues involve incorrect handling of command headers, authentication mechanisms, or non-volatile (NV) memory attributes.

Some examples:

- **Command header parsing errors:** Some TPMs did not correctly validate the `tag`, `commandSize`, or `commandCode` fields, which could allow crafted input data to crash the TPM or leak information.
- **Authorization bypass:** Certain firmware versions accepted commands without proper authorization or session checks.
- **Incorrect NV attribute handling:** Improper checks on NV index attributes (`written`, `readLocked`, `writeLocked`) caused inconsistent states or privilege escalation.
- **Side-channel leaks:** Timing differences and power analysis could be used to infer secret information from cryptographic operations.
- **Random number generator weaknesses:** Incomplete entropy initialization or poor DRBG seeding reduced key strength.

In addition to these general weaknesses, two notable advisories published by the Trusted Computing Group (TCG) illustrate common pitfalls in TPM firmware implementations:

- **TCGVRT0007 — Buffer overflow in TPM2_MakeCredential and TPM2_ActivateCredential (10):** Certain TPM 2.0 implementations failed to properly check buffer lengths during credential unmarshaling, allowing crafted credential blobs to cause memory corruption. This flaw could lead to a denial of service or, in specific cases, information leakage from uninitialized memory. The issue stemmed from unsafe memory copying and inadequate boundary validation during command decoding. *Mitigation:* Later firmware releases added size checks before decoding external inputs and enforced stricter limits on marshaled data lengths.
- **VRT0009 — Out-of-bounds read in TPM2_Startup and TPM2_Shutdown (11):** Improper parameter validation allowed out-of-range data to be read from internal buffers, potentially disclosing internal state or causing a TPM reset. While not directly exploitable for privilege escalation, it highlighted the risk of incomplete parameter sanitization in command handlers. *Mitigation:* Updated versions now enforce strict input bounds and clear unused memory regions after processing.

These examples emphasize the importance of strict input validation, safe unmarshaling routines, and consistent memory handling practices in any TPM implementation. Most vendors released firmware updates or new specifications to fix these problems by enforcing stronger checks, improved cryptography, and better state management.

5.2 Mitigated Vulnerabilities

Our TPM is robust against a wide set of **common CWE vulnerabilities**: by allowing a strict set of commands and even stricter formatting for parameters. We implement software-side tests with malicious commands to test the resilience of our implementation:

Command Tested	RC	Result / Description
TPM2_GetRandom	TPM_RC_SUCCESS (0x0000)	Random data (32 bytes) correctly returned, confirming proper TPM baseline functionality.
TPM2_CreatePrimary	TPM_RC_SUCCESS (0x0000)	RSA key created successfully at handle 0x80000001, validating cryptographic subsystem operation.
TPM2_NV_DefineSpace	TPM_RC_SUCCESS (0x0000)	NV index 0x01000001 (32 bytes) defined successfully, confirming proper initialization of persistent storage.
TPM2_NV_Write	TPM_RC_SUCCESS (0x0000)	Successfully wrote “TEST” to NV index; AES encryption and NV persistence path verified.
TPM2_NV_Read	TPM_RC_SUCCESS (0x0000)	Successfully decrypted and read “TEST”; confirms correct NV read/decrypt functionality.
BAD_HDR	TPM_RC_COMMAND_SIZE (0x0142)	Header claimed 48 bytes but only 12 were sent. TPM correctly detected mismatch and aborted parsing — prevents malformed packet parsing (CWE-130).
GET_RANDOM_TOO_BIG	TPM_RC_SIZE (0x0095)	Request for 65535 bytes exceeded valid range. TPM safely rejected request — integer bounds properly enforced (CWE-190).
NV_READ_OVERSIZE	TPM_RC_NV_RANGE	Attempted to read 1024 bytes from a 32-byte NV index. TPM blocked access — out-of-bounds protection confirmed (CWE-125).
NV_WRITE_OVERSIZE	TPM_RC_NV_RANGE	Write header mismatch (61 vs 56 bytes). TPM rejected operation early — prevents overflow in NV region (CWE-787).
NV_WRITE_FMT_STR	TPM_RC_COMMAND_SIZE (0x0142)	Command truncated; TPM stopped safely. No format string misuse observed in logs — logging path robust (CWE-134).
NV_READ_UNINIT	TPM_RC_HANDLE (0x008B)	NV index 0x01000002 undefined. TPM correctly returned handle-not-found — prevents uninitialized memory exposure (CWE-457).
INT_OVERFLOW	TPM_RC_SIZE (0x0095)	Oversized random request safely rejected. Confirms integer overflow and size validation mechanisms (CWE-190).

Table 5.1: Summary of TPM command tests, expected return codes, and interpreted results.

5.3 Unmitigated Vulnerabilities

During development and testing of our TPM module, several potential weaknesses were identified. Although these are mainly software-related and affect only the simulated environment, they highlight important areas to secure in a real TPM implementation.

- **Missing NV cleanup and lifecycle control:** The NV subsystem currently initializes and destroys state correctly during startup and shutdown, but the reset path does not invoke the cleanup routine. As a result, NV index data, lock flags, and internal bookkeeping structures may survive a soft reset, leaving behind orphaned entries or stale authorization state. This behavior deviates from the TPM lifecycle model, where **Startup(CLEAR)** is expected to reset volatile NV attributes and erase transient metadata. Without explicit cleanup in the

reset path, the emulator can enter inconsistent NV states across reset cycles, potentially preventing re-definition of indices or preserving sensitive state longer than intended.

- **Deterministic key derivation misuse:** The `tpm2_CreatePrimary()` implementation performs basic hierarchy validation but continues execution even when key derivation or elliptic-curve operations fail internally. Specifically, return values from `TPM2_KDFa()` and EC point initialization/multiplication routines are ignored, allowing the TPM to produce a primary object even if its seed-derived material was never generated correctly. This may yield invalid or inconsistent public/private pairs, undermining the trust guarantees associated with hierarchy-derived keys. In a real TPM, failures in KDFa or EC arithmetic would result in an explicit error response rather than silently constructing a malformed object, so the current behavior represents a deviation from expected cryptographic robustness and error handling discipline.
- **Unsafe logging and debug output:** Logging primitives `TPM2_LOG()` and `qemu_log()` are still affected by format string and buffer overflow vulnerabilities.

These fixes make the TPM emulator more stable and secure, preventing NV corruption, key errors, and unsafe memory use. While limited to the software model, they highlight essential properties a real TPM must guarantee — reliable key handling, strong randomness, and protected NV storage.

5.4 Future Work

There are several improvements that could make the TPM more complete and secure:

- **Platform Configuration Registers (PCRs)** — Implement PCRs to support platform attestation and measured boot. Enables integrity verification of firmware and software components during startup.
- **Non-Volatile Memory (NV) Enhancements** — Extend NV memory emulation with persistent state handling, and fine-grained access control. Improves realism and robustness for commands like `NV_DefineSpace` and `NV_Write`.
- **Fuzzing and Negative Testing** — Integrate fuzz testing for TPM command parsers to detect edge-case bugs, malformed input handling issues, and potential security flaws before deployment.
- **Cryptographic Protections** — Strengthen the TPM’s cryptographic layer by adding side-channel and timing attack mitigations. Introduce new primitives such as ECC and stronger random number generation for modern security standards.
- **Continuous Integration and Analysis** — Set up automated build pipelines so that errors are caught early.

These steps would help the TPM simulator better reflect real hardware behavior and improve its resilience against both logical and low-level attacks.

CHAPTER 6

TPM Software Testing

This chapter addresses tests done to ensure correct the implemented TPM 2.0 functionality.

6.1 Firmware Testing

The firmware's `main.c` prints an interactive menu for the TPM MMIO interface. Each menu option sends a prebuilt TPM 2.0 command, validates the response header, and prints detailed diagnostics over UART.

6.1.1 Driving TPM Commands from the Console

Listing 6.1: Console menu dispatching TPM command buffers

```
1 switch (c) {
2 case '1':
3     cmd_to_send = CMD_GET_RANDOM;
4     cmd_size     = sizeof(CMD_GET_RANDOM);
5     break;
6 case '2':
7     cmd_to_send = CMD_CREATE_PRIMARY;
8     cmd_size     = sizeof(CMD_CREATE_PRIMARY);
9     break;
10 case '3':
11     cmd_to_send = CMD_NV_DEFINE_SPACE;
12     cmd_size     = sizeof(CMD_NV_DEFINE_SPACE);
13     break;
14 default:
15     UART_printf("Invalid option '%c'\n", c);
16     cmd_to_send = NULL;
17     break;
18 }
```

What it verifies. Each command covers a different back-end subsystem—random generation, ECC key creation, and NV definition. Choosing an option streams the corresponding big-endian buffer through the MMIO FIFO, confirming that padding, word alignment, and the control handshake remain valid after code changes.

6.1.2 Checking Response Integrity

Listing 6.2: Reading and printing TPM response from MMIO FIFO

```
1 uint32_t resp_size = tpm_read_response(response_buffer,
2                                     sizeof(response_buffer));
```

```

3
4 if (resp_size > 0) {
5     parse_response_header(response_buffer);
6     UART_printf("\nFull Response (Hex):\n");
7     UART_print_hex(response_buffer, resp_size);
8 } else {
9     UART_printf("Failed to get response.\n");
10 }

```

What it verifies. After triggering a command, the firmware waits until the TPM enters the SENDING state, enforces size limits, and parses the 10-byte header to confirm the return code is sensible. Any malformed header aborts the read, making the console an immediate regression detector for protocol changes.

6.2 Back-End Realization Tests

When the TPM device is realized inside QEMU (tpm_hw.c), it runs internal tests to validate the NV subsystem, key derivation logic, and symmetric encryption stack before the guest sees the device.

6.2.1 Sanity-Checking NV Allocation

Listing 6.3: Realization-time TPM2_NV_DefineSpace smoke test

```

1 static void tpm2_test_definespace(TPM2State *s)
2 {
3     TPM2B_AUTH auth = { .size = 4, .buffer = {'1','2','3','4'} };
4     TPMS_NV_PUBLIC pub = { ... .dataSize = 32 };
5     TPM2B_NV_PUBLIC publicInfo = { .size = sizeof(pub), .nvPublic = pub };
6
7     TPM_RC rc = tpm2_nv_define_space(s, tpm2_provision_owner(),
8                                     &auth, &publicInfo);
9
10    if (rc == TPM_RC_SUCCESS)
11        TPM2_LOG("[INIT] NV DefineSpace success (count=%u)\n", s->nv_count);
12    else
13        TPM2_LOG("[INIT] NV DefineSpace failed! RC=0x%X\n", rc);
14 }

```

What it verifies. Immediately after NV initialization, the device attempts to allocate an ordinary NV index. Success proves that the NV bank is sized correctly, the attribute validators accept legal combinations, and the GLib hash table bookkeeping is intact.

6.2.2 Exercising Deterministic Key Creation

Listing 6.4: Deterministic TPM2_CreatePrimary test during initialization

```

1 TPM_RC tpm2_test_CreatePrimary(TPM2State *s)
2 {
3     for (int i = 0; i < 32; i++) {
4         s->sps[i] = 0x10 + i;
5         s->pps[i] = 0x30 + i;
6         ...
7     }

```

```

8   TPM2B_SENSITIVE_CREATE inSensitive = { ... };
9   TPM2B_PUBLIC inPublic
10      = { ... TPM_ALG_ECC ... };
11
12   TPM_RC rc = tpm2_CreatePrimary(s, TPM_RH_OWNER,
13                                  &inSensitive, &inPublic,
14                                  NULL, NULL, NULL,
15                                  &outPublic, &name);
16
17   if (rc != TPM_RC_SUCCESS) {
18       TPM2_LOG("FAILED: rc=0x%X\n", rc);
19       return rc;
20   }
21   ...
22   return TPM_RC_SUCCESS;

```

What it verifies. By deriving a primary ECC key during realization, the device proves that hierarchy seeds are initialised, KDFa succeeds, the curve math produces affine coordinates, and the marshalled public area is coherent—critical prerequisites before the firmware attempts the same command.

6.2.3 Round-Trip NV Encryption

Listing 6.5: Encrypted NV write + decrypt readback self-test

```

1  static void tpm2_test_nv_encrypt_decrypt(TPM2State *s)
2  {
3      NVEntry *e = g_hash_table_lookup(s->nv_map,
4                                         GUIN_T0_POINTER(0x1500016));
5
6      if (!e) {
7          TPM2_LOG("[TEST] NV index not found\n");
8          return;
9      }
10
11     const char *msg = "TPM NV TEST DATA";
12     TPM_RC rc = TPM2_NV_Write(s, e,
13                               (const uint8_t *)msg,
14                               strlen(msg), 0);
15
16     rc = nv_read_decrypt_from_bank(s, e, decrypted,
17                                    strlen(msg), 0);
18     ...
19 }

```

What it verifies. After the DefineSpace smoke test, the device encrypts a string into the newly created NV entry, dumps the ciphertext, and decrypts it back to confirm the AES-CFB wrapper uses fresh IVs and honours data length limits. Any failure warns the developer before the firmware touches NV storage.

6.3 Summary

Together, these self-tests provide immediate feedback—without compiling the dedicated host harness—that the firmware/TPM contract still holds. When extending functionality, mirror this pattern: add an interactive console path in the firmware and a realization-time probe in the device model so regressions surface early in the development cycle.

CHAPTER 7

Programmer's Manual

This chapter describes the software interface of the TPM 2.0 custom device implemented in QEMU

7.1 Memory-Mapped Interface

The TPM device is exposed as a `sysbus` peripheral at address `0xF0000000`. Table 7.1 lists the memory-mapped control and data registers.

Table 7.1: TPM MMIO register map

Offset	Register	Access	Description
0x00	TPM2_CTRL_REG	R/W	Controls TPM state transitions (Idle, Receiving, Processing)
0x04	TPM2_STATUS_REG	R	Returns current TPM state (Idle, Receiving, Sending)
0x08	TPM2_RANDOM_REG	R	Optional direct random output (unused in current version)
0x0C	TPM2_CMD_REG	W	Triggers command execution after all bytes are written
0x10	TPM2_DATA_REG	R/W	FIFO interface for command and response data

Each command follows the standard TPM 2.0 request format defined in Part 3 of the specification

7.2 TPM Command Transmission

Firmware communicates with the TPM by writing command bytes to `TPM2_DATA_REG`, then triggering execution via `TPM2_CMD_REG`. The example below shows the helper function used to transmit a command buffer to the TPM device.

Listing 7.1: Sending a TPM command from firmware

```
1 void tpm_send_command(const uint8_t *cmd_buf, uint32_t len)
2 {
3     TPM2_CTRL_REG = 1; // Enter RECEIVING mode
4     uint32_t padded_len = (len + 3) & ~3;
5
6     for (uint32_t i = 0; i < padded_len; i += 4) {
7         uint32_t word = 0;
8         if (i + 0 < len) word |= cmd_buf[i + 0] << 24;
9         if (i + 1 < len) word |= cmd_buf[i + 1] << 16;
10        if (i + 2 < len) word |= cmd_buf[i + 2] << 8;
11        if (i + 3 < len) word |= cmd_buf[i + 3];
12        TPM2_DATA_REG = word; // Write 4 bytes at a time
13    }
14 }
```

```

15     TPM2_CMD_REG = 1; // Execute command
16 }

```

7.3 Reading TPM Responses

Once the command has been processed, the TPM enters `SENDING` state, allowing the firmware to read back the response data from the same data register.

Listing 7.2: Reading a TPM response

```

1  uint32_t tpm_read_response(uint8_t *resp_buf, uint32_t max_len)
2  {
3      while (TPM2_STATUS_REG == TPM_STATE_PROCESSING) { /* wait */ }
4
5      if (TPM2_STATUS_REG != TPM_STATE_SENDING)
6          return 0;
7
8      for (int i = 0; i < 3; i++) {
9          uint32_t word = TPM2_DATA_REG;
10         resp_buf[i*4+0] = (word >> 24) & 0xFF;
11         resp_buf[i*4+1] = (word >> 16) & 0xFF;
12         resp_buf[i*4+2] = (word >> 8) & 0xFF;
13         resp_buf[i*4+3] = (word) & 0xFF;
14     }
15
16     uint32_t size = ((uint32_t)resp_buf[2] << 24) |
17                    ((uint32_t)resp_buf[3] << 16) |
18                    ((uint32_t)resp_buf[4] << 8) |
19                    ((uint32_t)resp_buf[5]);
20     return size;
21 }

```

7.4 Built-in TPM Commands

The following TPM 2.0 commands are currently supported in the simulator. Each command can be triggered directly by selecting the corresponding menu option in `main.c`.

- **TPM2_GetRandom**: Generates a random sequence of bytes using OpenSSL’s `RAND_bytes()` function. Returns the requested number of bytes in the response body.
- **TPM2_CreatePrimary**: Creates a temporary ECC primary key. The QEMU backend (`tpm2_generate_rsa_key()`) builds the keypair using OpenSSL and returns a minimal public structure.
- **TPM2_NV_DefineSpace**: Defines a new NV index and allocates a corresponding entry in the TPM’s internal NV bank. Attributes and data size are parsed and stored in a hash map managed by the backend.
- **TPM2_NV_Write**: Writes data into an NV index with encryption applied transparently by the TPM backend. Enforces NV permissions, offset and size rules, and attribute constraints before encrypting and committing the buffer to the internal NV store.
- **nv_read_decrypt_from_bank**: Reads data back from an NV index and decrypts it using the same seed-derived key. Ensures bounds checking and returns plaintext to the caller, validating the round-trip confidentiality path for NV-protected data.

7.5 Example: Sending TPM2_GetRandom

The example below shows a complete command-response cycle using the prebuilt `CMD_GET_RANDOM` buffer.

Listing 7.3: Example: TPM2_GetRandom command usage

```
1 uint8_t resp_buf[64];
2 UART_printf("Requesting 32 random bytes...\n");
3 tpm_send_command(CMD_GET_RANDOM, sizeof(CMD_GET_RANDOM));
4 uint32_t resp_size = tpm_read_response(resp_buf, sizeof(resp_buf));
5 UART_print_hex(resp_buf, resp_size);
```

7.6 TPM State Structures

The TPM backend maintains its operational state inside the `TPM2State` structure

Table 7.2: Selected fields of `TPM2State`

Field	Type	Description
uint8_t sps[32]	array	Owner hierarchy seed (temporary)
TPM2B_SENSITIVE primary_sensitive	struct	Primary key sensitive area
GHashTable *nv_map	pointer	Maps NV indices to entries (NVEnter*)
uint8_t *nv_bank_ptr	pointer	Base address of NV memory region
RSA *rsa_key	pointer	Current primary RSA key (if generated)
uint8_t command_buffer[4096]	array	Buffer for incoming TPM commands
uint8_t response_buffer[4096]	array	Buffer for outgoing TPM responses

7.7 NV Bank Organization

Each NV entry is represented by the following structure, allocated dynamically by `nv_entry_alloc()`:

Listing 7.4: NVEnter structure (simplified)

```
1 typedef struct NVEnter {
2     TPMS_NV_PUBLIC pub;    // NV metadata
3     uint8_t *iv_ptr;      // 16-byte AES IV
4     uint8_t *data;        // Encrypted NV data
5     uint16_t dataLen;
6     bool written;
7     bool readLocked;
8     bool writeLocked;
9 } NVEnter;
```

The NV region is backed by a private memory block (`nv_bank_mem`), which is not exposed to the guest. Each NV entry reserves a 16-byte IV followed by its encrypted data payload.

7.8 Initialization Sequence

When the TPM device is realized in QEMU, the following steps occur automatically:

1. NV bank memory is allocated and cleared.
2. The NV subsystem is initialized via `tpm2_nv_init()`.
3. A test NV index is created (`tpm2_test_definespace()`).
4. A primary ECC key is derived (`tpm2_test_CreatePrimary()`).
5. NV encryption/decryption is verified (`tpm2_test_nv_encrypt_decrypt()`).

This ensures that all major TPM subsystems are functional immediately upon startup.

7.9 Summary

Developers can extend this framework by adding additional TPM 2.0 commands to `tpm2_process_command()`, or by modifying NV management and cryptographic functions for testing purposes. All functions are fully logged to QEMU's output, aiding debugging and verification.

Bibliography

- [1] Q. Project, “Qemu documentation,” <https://www.qemu.org/documentation/>, accessed: 2025-10-19.
- [2] Quarkslab, “Taming the qemu beast: An overview of qemu’s internals,” <https://github.com/quarkslab/sstic-tame-the-qemu>, 2017, accessed: 2025-10-28.
- [3] T. C. Group, “Tpm 2.0 library specification, part 0: Overview, version 1.84,” Trusted Computing Group, Tech. Rep., 2020. [Online]. Available: https://trustedcomputinggroup.org/wp-content/uploads/Trusted-Platform-Module-2.0-Library-Part-0-Version-184_pub.pdf
- [4] —, “Tpm 2.0 library specification, part 2: Structures, version 1.83,” Trusted Computing Group, Tech. Rep., 2020. [Online]. Available: <https://trustedcomputinggroup.org/wp-content/uploads/TPM-2.0-1.83-Part-2-Structures.pdf>
- [5] —, “Tpm 2.0 library specification, part 3: Commands, version 1.84,” Trusted Computing Group, Tech. Rep., 2020. [Online]. Available: https://trustedcomputinggroup.org/wp-content/uploads/Trusted-Platform-Module-2.0-Library-Part-3-Version-184_pub.pdf
- [6] —, “Tpm 2.0 provisioning guidance, version 1.1,” Trusted Computing Group, Tech. Rep., 2018. [Online]. Available: <https://trustedcomputinggroup.org/wp-content/uploads/TCG-TPM-v2.0-Provisioning-Guidance-Published-v1r1.pdf>
- [7] O. Project, “Openssl github repository,” <https://github.com/openssl/openssl>, accessed: 2025-10-28.
- [8] —, “Openssl official documentation,” <https://docs.openssl.org/master/>, accessed: 2025-10-28.
- [9] T. C. Group, “Tpm 2.0 library specification,” *TCG Specification*, 2020. [Online]. Available: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>
- [10] —, “Tcgvrt0007 advisory: Buffer overflow in tpm2_makecredential and tpm2_activatecredential,” Trusted Computing Group (TCG) Vulnerability Response Team, Tech. Rep., 2023, accessed: 2025-10-28. [Online]. Available: <https://trustedcomputinggroup.org/wp-content/uploads/TCGVRT0007-Advisory-FINAL.pdf>
- [11] —, “Vrt0009 advisory: Out-of-bounds read in tpm2_startup and tpm2_shutdown,” Trusted Computing Group (TCG) Vulnerability Response Team, Tech. Rep., 2023, accessed: 2025-10-28. [Online]. Available: <https://trustedcomputinggroup.org/wp-content/uploads/VRT0009-Advisory-FINAL.pdf>